

# Quantum Computing, Quantum Machine Learning and Quantum Information Theories

Morten Hjorth-Jensen<sup>1,2</sup>

Department of Physics, University of Oslo<sup>1</sup>

Department of Physics and Astronomy and Facility for Rare Isotope Beams,  
Michigan State University, USA<sup>2</sup>

February 12, 2025

© 1999-2025, Morten Hjorth-Jensen. Released under CC Attribution-NonCommercial 4.0 license

## Plans for the week of February 10-14

1. Reminder from last week on gates and circuits
2. One-qubit and two-qubit gates, background and realizations
3. Simple Hamiltonian systems
4. Video of lecture to be added

# Readings

1. For the discussion of one-qubit, two-qubit and other gates, sections 2.6-2.11 and 3.1-3.4 of Hundt's book **Quantum Computing for Programmers**, contain most of the relevant information.

## Gates, the whys and hows

In quantum computing it is common to rewrite various unitary transformations acting on a given state, in terms of so-called gates (one-qubit, two-qubit or more qubit gates). These unitary transformations do actually represent specific interactions of the system with the environment.

Each such operation is by convention written in terms of gates and a chain of such gates represents a circuit. The latter represents then a specific set of operations on an initial state in order to perform what we will label as experiments.

The aim of the first set of notes this week is to link these gates (and thereby circuits) to their respective unitary transformation. These unitary transformations represent selected physical processes.

# Structure of the lecture

1. First we review some of the basic ways of representing the solution to the Schrödinger equation, introducing the so-called Interaction, Heisenberg and Schrödinger pictures and unitary transformations.
2. Secondly, we present examples of physical processes and how they can be represented as unitary operations on a given state.
3. These unitary transformations are then represented as gates. Setting gates together gives us a final circuit which can represent a specific physical system

## Part 1: Mathematical background

The time-dependent Schrödinger equation (or equation of motion) reads

$$i\hbar \frac{\partial}{\partial t} |\Psi_S(t)\rangle = \hat{H} \Psi_S(t)\rangle,$$

where the subscript  $S$  stands for Schrödinger here. A formal solution is given by

$$|\Psi_S(t)\rangle = \exp(-i\hat{H}(t - t_0)/\hbar) |\Psi_S(t_0)\rangle.$$

# Unitary transformation

The Hamiltonian  $\hat{H}$  is hermitian and the exponent represents a unitary operator with an operation carried out on the wave function at a time  $t_0$ .

The exponential term  $\exp(-i\hat{H}(t - t_0)/\hbar)$  is our unitary transformation  $U(t, t_0)$  and we have

$$|\Psi_S(t)\rangle = \exp(-i\hat{H}(t - t_0)/\hbar)|\Psi_S(t_0)\rangle = U(t, t_0)|\Psi_S(t_0)\rangle.$$

## Interaction picture

Our Hamiltonian is normally written out as the sum of an unperturbed part  $\hat{H}_0$  and an interaction part  $\hat{H}_I$ , that is

$$\hat{H} = \hat{H}_0 + \hat{H}_I.$$

In general we have  $[\hat{H}_0, \hat{H}_I] \neq 0$  since  $[\hat{T}, \hat{V}] \neq 0$ . We wish now to define a unitary transformation in terms of  $\hat{H}_0$  by defining

$$|\Psi_I(t)\rangle = \exp(i\hat{H}_0 t/\hbar) |\Psi_S(t)\rangle,$$

which is again a unitary transformation carried out now at the time  $t$  on the wave function in the Schrödinger picture.



## Taking the derivative wrt time

We can easily find the equation of motion by taking the time derivative

$$i\hbar \frac{\partial}{\partial t} |\Psi_I(t)\rangle = -\hat{H}_0 \exp(i\hat{H}_0 t/\hbar) \Psi_S(t) + \exp(i\hat{H}_0 t/\hbar) i\hbar \frac{\partial}{\partial t} \Psi_S(t).$$

## Expression using the interaction picture

Using the definition of the Schrödinger equation, we can rewrite the last equation as

$$i\hbar \frac{\partial}{\partial t} |\Psi_I(t)\rangle = \exp(i\hat{H}_0 t/\hbar) \left[ -\hat{H}_0 + \hat{H}_0 + \hat{H}_I \right] \exp(-i\hat{H}_0 t/\hbar) \Psi_I(t)\rangle,$$

which gives us

$$i\hbar \frac{\partial}{\partial t} |\Psi_I(t)\rangle = \hat{H}_I(t) \Psi_I(t)\rangle,$$

with

$$\hat{H}_I(t) = \exp(i\hat{H}_0 t/\hbar) \hat{H}_I \exp(-i\hat{H}_0 t/\hbar).$$

## Expectation value

The order of the operators is important since  $\hat{H}_0$  and  $\hat{H}_I$  do generally not commute. The expectation value of an arbitrary operator in the interaction picture can now be written as

$$\langle \Psi'_S(t) | \hat{O}_S | \Psi_S(t) \rangle = \langle \Psi'_I(t) | \exp(i\hat{H}_0 t/\hbar) \hat{O}_I \exp(-i\hat{H}_0 t/\hbar) | \Psi_I(t) \rangle,$$

and using the definition

$$\hat{O}_I(t) = \exp(i\hat{H}_0 t/\hbar) \hat{O}_I \exp(-i\hat{H}_0 t/\hbar),$$

we obtain

$$\langle \Psi'_S(t) | \hat{O}_S | \Psi_S(t) \rangle = \langle \Psi'_I(t) | \hat{O}_I(t) | \Psi_I(t) \rangle,$$

stating that a unitary transformation does not change expectation values!

## Interaction picture and equation of motion

If we take the time derivative of the operator in the interaction picture we arrive at the following equation of motion

$$i\hbar \frac{\partial}{\partial t} \hat{O}_I(t) = \exp(i\hat{H}_0 t/\hbar) \left[ \hat{O}_S \hat{H}_0 - \hat{H}_0 \hat{O}_S \right] \exp(-i\hat{H}_0 t/\hbar) = \left[ \hat{O}_I(t), \hat{H}_0 \right]$$

Here we have used the time-independence of the Schrödinger equation together with the observation that any function of an operator commutes with the operator itself.

## Unitary operator

In order to solve the equation of motion equation in the interaction picture, we define a unitary time-development operator  $\hat{U}(t, t')$ . Later we will derive its connection with the linked-diagram theorem, which yields a linked expression for the actual operator. The action of the operator on the wave function is

$$|\Psi_I(t)\rangle = \hat{U}(t, t_0)|\Psi_I(t_0)\rangle,$$

with the obvious value  $\hat{U}(t_0, t_0) = 1$ .

## Time-development operator

The time-development operator  $U$  has the properties that

$$\hat{U}^\dagger(t, t')\hat{U}(t, t') = \hat{U}(t, t')\hat{U}^\dagger(t, t') = 1,$$

which implies that  $U$  is unitary

$$\hat{U}^\dagger(t, t') = \hat{U}^{-1}(t, t').$$

Further,

$$\hat{U}(t, t')\hat{U}(t', t'') = \hat{U}(t, t'')$$

and

$$\hat{U}(t, t')\hat{U}(t', t) = 1,$$

which leads to

$$\hat{U}(t, t') = \hat{U}^\dagger(t', t).$$

## Properties of the operator $U$

Using our definition of Schrödinger's equation in the interaction picture, we can then construct the operator  $\hat{U}$ . We have defined

$$|\Psi_I(t)\rangle = \exp(i\hat{H}_0 t/\hbar)|\Psi_S(t)\rangle,$$

which can be rewritten as

$$|\Psi_I(t)\rangle = \exp(i\hat{H}_0 t/\hbar) \exp(-i\hat{H}(t - t_0)/\hbar)|\Psi_S(t_0)\rangle,$$

or

$$|\Psi_I(t)\rangle = \exp(i\hat{H}_0 t/\hbar) \exp(-i\hat{H}(t - t_0)/\hbar) \exp(-i\hat{H}_0 t_0/\hbar)|\Psi_I(t_0)\rangle.$$

## Interaction picture

From the last expression we can define

$$\hat{U}(t, t_0) = \exp(i\hat{H}_0 t/\hbar) \exp(-i\hat{H}(t - t_0)/\hbar) \exp(-i\hat{H}_0 t_0/\hbar).$$

It is then easy to convince oneself that the properties defined above are satisfied by the definition of  $\hat{U}$ .



## Equation of motion

We derive the equation of motion for  $\hat{U}$  using the above definition. This results in

$$i\hbar \frac{\partial}{\partial t} \hat{U}(t, t_0) = \hat{H}_I(t) \hat{U}(t, t_0),$$

which we integrate from  $t_0$  to a time  $t$  resulting in

$$\hat{U}(t, t_0) - \hat{U}(t_0, t_0) = \hat{U}(t, t_0) - 1 = -\frac{i}{\hbar} \int_{t_0}^t dt' \hat{H}_I(t') \hat{U}(t', t_0),$$

which can be rewritten as

$$\hat{U}(t, t_0) = 1 - \frac{i}{\hbar} \int_{t_0}^t dt' \hat{H}_I(t') \hat{U}(t', t_0).$$

# Time-ordering

We can solve this equation iteratively keeping in mind the time-ordering of the operators

$$\hat{U}(t, t_0) = 1 - \frac{i}{\hbar} \int_{t_0}^t dt' \hat{H}_I(t') + \left(\frac{-i}{\hbar}\right)^2 \int_{t_0}^t dt' \int_{t_0}^{t'} dt'' \hat{H}_I(t') \hat{H}_I(t'') + \dots$$

The third term can be written as

$$\begin{aligned} \int_{t_0}^t dt' \int_{t_0}^{t'} dt'' \hat{H}_I(t') \hat{H}_I(t'') &= \frac{1}{2} \int_{t_0}^t dt' \int_{t_0}^{t'} dt'' \hat{H}_I(t') \hat{H}_I(t'') \\ &\quad + \frac{1}{2} \int_{t_0}^t dt'' \int_{t''}^t dt' \hat{H}_I(t') \hat{H}_I(t''). \end{aligned}$$

## Changing order of integrations

We obtain this expression by changing the integration order in the second term via a change of the integration variables  $t'$  and  $t''$  in

$$\frac{1}{2} \int_{t_0}^t dt' \int_{t_0}^{t'} dt'' \hat{H}_I(t') \hat{H}_I(t'').$$

We can rewrite the terms which contain the double integral as

$$\int_{t_0}^t dt' \int_{t_0}^{t'} dt'' \hat{H}_I(t') \hat{H}_I(t'') =$$

$$\frac{1}{2} \int_{t_0}^t dt' \int_{t_0}^{t'} dt'' \left[ \hat{H}_I(t') \hat{H}_I(t'') \Theta(t' - t'') + \hat{H}_I(t') \hat{H}_I(t'') \Theta(t'' - t') \right],$$

with  $\Theta(t'' - t')$  being the standard Heavyside or step function. The step function allows us to give a specific time-ordering to the above expression.

## Time-ordering operator

With the  $\Theta$ -function we can rewrite the last expression as

$$\int_{t_0}^t dt' \int_{t_0}^{t'} dt'' \hat{H}_I(t') \hat{H}_I(t'') = \frac{1}{2} \int_{t_0}^t dt' \int_{t_0}^{t'} dt'' \hat{T} \left[ \hat{H}_I(t') \hat{H}_I(t'') \right],$$

where  $\hat{T}$  is the so-called time-ordering operator.

## Final expression for $\hat{U}$

With this definition, we can rewrite the expression for  $\hat{U}$  as

$$\hat{U}(t, t_0) = \sum_{n=0}^{\infty} \left( \frac{-i}{\hbar} \right)^n \frac{1}{n!} \int_{t_0}^t dt_1 \cdots \int_{t_0}^t dt_N \hat{T} \left[ \hat{H}_I(t_1) \cdots \hat{H}_I(t_n) \right] =$$
$$\hat{T} \exp \left[ \frac{-i}{\hbar} \int_{t_0}^t dt' \hat{H}_I(t') \right].$$

## Heisenberg picture as alternative

We wish now to define a unitary transformation in terms of  $\hat{H}$  by defining

$$|\Psi_H(t)\rangle = \exp(i\hat{H}t/\hbar)|\Psi_S(t)\rangle,$$

which is again a unitary transformation carried out now at the time  $t$  on the wave function in the Schrödinger picture. If we combine this equation with Schrödinger's equation we obtain the following equation of motion

$$i\hbar \frac{\partial}{\partial t} |\Psi_H(t)\rangle = 0,$$

meaning that  $|\Psi_H(t)\rangle$  is time independent. An operator in this picture is defined as

$$\hat{O}_H(t) = \exp(i\hat{H}t/\hbar) \hat{O}_S \exp(-i\hat{H}t/\hbar).$$

## New equation of motion

The time dependence is then in the operator itself, and this yields in turn the following equation of motion

$$i\hbar \frac{\partial}{\partial t} \hat{O}_H(t) = \exp(i\hat{H}t/\hbar) \left[ \hat{O}_H \hat{H} - \hat{H} \hat{O}_H \right] \exp(-i\hat{H}t/\hbar) = \left[ \hat{O}_H(t), \hat{H} \right].$$

We note that an operator in the Heisenberg picture can be related to the corresponding operator in the interaction picture as

$$\begin{aligned} \hat{O}_H(t) &= \exp(i\hat{H}t/\hbar) \hat{O}_S \exp(-i\hat{H}t/\hbar) = \\ &\exp(i\hat{H}_I t/\hbar) \exp(-i\hat{H}_0 t/\hbar) \hat{O}_I \exp(i\hat{H}_0 t/\hbar) \exp(-i\hat{H}_I t/\hbar). \end{aligned}$$

## Unitary transformations again

With our definition of the time evolution operator we see that

$$\hat{O}_H(t) = \hat{U}(0, t) \hat{O}_I \hat{U}(t, 0),$$

which in turn implies that  $\hat{O}_S = \hat{O}_I(0) = \hat{O}_H(0)$ , all operators are equal at  $t = 0$ . The wave function in the Heisenberg formalism is related to the other pictures as

$$|\Psi_H\rangle = |\Psi_S(0)\rangle = |\Psi_I(0)\rangle,$$

since the wave function in the Heisenberg picture is time independent. We can relate this wave function to that at a given time  $t$  via the time evolution operator as

$$|\Psi_H\rangle = \hat{U}(0, t) |\Psi_I(t)\rangle.$$



## Adiabatic switching

We assume that the interaction term is switched on gradually. Our wave function at time  $t = -\infty$  and  $t = \infty$  is supposed to represent a non-interacting system given by the solution to the unperturbed part of our Hamiltonian  $\hat{H}_0$ . We assume the ground state is given by  $|\Phi_0\rangle$ , which could be a Slater determinant. We define our Hamiltonian as

$$\hat{H} = \hat{H}_0 + \exp(-\varepsilon t/\hbar)\hat{H}_I,$$

where  $\varepsilon$  is a small number. The way we write the Hamiltonian and its interaction term is meant to simulate the switching of the interaction.

## Time evolution

The time evolution of the wave function in the interaction picture is then

$$|\Psi_I(t)\rangle = \hat{U}_\varepsilon(t, t_0)|\Psi_I(t_0)\rangle,$$

with

$$\begin{aligned} \hat{U}_\varepsilon(t, t_0) = & \sum_{n=0}^{\infty} \left( \frac{-i}{\hbar} \right)^n \frac{1}{n!} \int_{t_0}^t dt_1 \cdots \int_{t_0}^t dt_N \\ & \times \exp(-\varepsilon(t_1 + \cdots + t_n)/\hbar) \hat{T} \left[ \hat{H}_I(t_1) \cdots \hat{H}_I(t_n) \right] \end{aligned}$$

## Initial state preparation

In the limit  $t_0 \rightarrow -\infty$ , the solution of Schrödinger's equation is  $|\Phi_0\rangle$ , and the eigenenergies are given by

$$\hat{H}_0|\Phi_0\rangle = W_0|\Phi_0\rangle,$$

meaning that

$$|\Psi_S(t_0)\rangle = \exp(-iW_0t_0/\hbar)|\Phi_0\rangle,$$

with the corresponding interaction picture wave function given by

$$|\Psi_I(t_0)\rangle = \exp(i\hat{H}_0t_0/\hbar)|\Psi_S(t_0)\rangle = |\Phi_0\rangle.$$

## Final expression

The solution becomes time independent in the limit  $t_0 \rightarrow -\infty$ .  
The same conclusion can be reached by looking at

$$i\hbar \frac{\partial}{\partial t} |\Psi_I(t)\rangle = \exp(\varepsilon|t|/\hbar) \hat{H}_I |\Psi_I(t)\rangle$$

and taking the limit  $t \rightarrow -\infty$ . We can rewrite the equation for the wave function at a time  $t = 0$  as

$$|\Psi_I(0)\rangle = \hat{U}_\varepsilon(0, -\infty) |\Phi_0\rangle.$$

## Part 2: Specific realizations and famous gates

Nuclear magnetic resonance (NMR) quantum computing is one of the several proposed approaches for constructing a quantum computer. It uses the spin states of nuclei within molecules as qubits. The quantum states are probed through the nuclear magnetic resonances, allowing the system to be implemented as a variation of nuclear magnetic resonance spectroscopy. NMR differs from other implementations of quantum computers in that it uses an ensemble of systems, in this case molecules, rather than a single pure state.

You can read more about this at

<https://cba.mit.edu/docs/papers/98.06.sciqc.pdf>

# Spin Hamiltonian

In order to understand in terms of a given Hamiltonian how the different gates arise, we consider now the Hamiltonian of a nuclear spin in a magnetic field. Since the spin provides provides a magnetic dipole moment, a nucleus with a spin will interact with the magnetic field. The Hamiltonian of a nucleus with spin interacting with a magnetic field  $\mathbf{B}$  is

$$H = -\boldsymbol{\mu}\mathbf{B},$$

with  $\boldsymbol{\mu} = \gamma\mathbf{S}$ ,  $\gamma$  being the so-called gyromagnetic ratio and  $\mathbf{S}$  the spin.

## Field along the z-axis

It is common to let the spin interact with a constant magnetic field along the z-axis. This gives an effective Hamiltonian

$$H_z = -\frac{\hbar\omega_L}{2}\sigma_z,$$

where  $\omega_L$  is the so-called Larmor precession frequency. This quantity includes also the constant magnetic field along the z-axis. For all practical purposes it suffices for us to have an expression of the Hamiltonian in terms of the Pauli-Z matrix.

## Bringing back a state on the Bloch sphere

Suppose that our initial one qubit state (for example a spin-1/2 nucleus for NMR studies) points along some arbitrary axis. As discussed during our second week, a point on the Bloch sphere can be represented as at time  $t = 0$

$$|\psi(t=0)\rangle = |\psi(0)\rangle = \cos\left(\frac{\theta}{2}\right)|0\rangle + \exp i\phi \sin\left(\frac{\theta}{2}\right)|1\rangle.$$



# Time evolution

Since the hamiltonian is time-independent, the state  $|\psi(0)\rangle$ , our system will evolve according to the unitary transformation

$$|\psi(t)\rangle = U(t)|\psi(0)\rangle = \exp i\omega_L t\sigma_z/2|\psi(0)\rangle.$$

Inserting the Bloch sphere ansatz we have then

$$|\psi(t)\rangle = \exp i\omega_L t\sigma_z/2 \cos(\frac{\theta}{2})|0\rangle + \exp i\omega_L t\sigma_z/2 \exp i\phi \sin(\frac{\theta}{2})|1\rangle.$$

The specific hamiltonian we have chosen here serves to exemplify how can represent physical operations in terms of specific gates, here a one-qubit gate (see whiteboard notes at <https://github.com/CompPhysics/QuantumComputingMachineLearning/blob/gh-pages/doc/HandWrittenNotes/2025/NotesFebruary12.pdf> for more details).

## Final expression

In exercise 4 from the second week, we showed that, given  $\mathbf{A}$  an operator on a vector space satisfying  $\mathbf{A}^2 = 1$  and  $\alpha$  any real constant, we had

$$\exp i\alpha \mathbf{A} = \sum_{n=0}^{\infty} \frac{(i\alpha)^n}{n!} \mathbf{A}^n = \mathbf{I} \cos \alpha + i\mathbf{A} \sin \alpha.$$

Using this result, we obtain

$$|\psi(t)\rangle = \exp i\omega_L t/2 \cos(\frac{\theta}{2})|0\rangle + \exp -i\omega_L t/2 \exp i\phi \sin(\frac{\theta}{2})|1\rangle.$$

The whiteboard notes for this week contain other examples of one qubit gates and their relation to specific unitary transformations and effective Hamiltonian, see <https://github.com/CompPhysics/QuantumComputingMachineLearning/blob/gh-pages/doc/HandWrittenNotes/2025/NotesFebruary12.pdf>

## Part 3: Famous Quantum gates, circuits and simple algorithms (repetition from last week)

Quantum gates are physical actions that are applied to the physical system representing the qubits. Mathematically, they are complex-valued, unitary matrices which act on the complex-valued normalized vectors that represent qubits. As the quantum analog of classical logic gates (such as AND and OR), there is a corresponding quantum gate for every classical gate; however, there are quantum gates that have no classical counter-part. They act on a set of qubits and, changing their state. That is, if  $U$  is a quantum gate and  $|q\rangle$  is a qubit, then acting the gate  $U$  on the qubit  $|q\rangle$  transforms the qubit as follows:

$$|q\rangle \xrightarrow{U} U|q\rangle. \quad (1)$$

# Quantum circuits

Quantum circuits are diagrammatic representations of quantum algorithms. The horizontal dimension corresponds to time; moving left to right corresponds to forward motion in time. They consist of a set of qubits  $|q_n\rangle$  which are stacked vertically on the left-hand side of the diagram. Lines, called quantum wires, extend horizontally to the right from each qubit, representing its state moving forward in time. Additionally, they contain a set of quantum gates that are applied to the quantum wires. Gates are applied chronologically, left to right.

# Single-Qubit Gates

A single-qubit gate is a physical action that is applied to one qubit. It can be represented by a matrix  $U$  from the group  $SU(2)$ . Any single-qubit gate can be parameterized by three angles:  $\theta$ ,  $\phi$ , and  $\lambda$  as follows

$$U(\theta, \phi, \lambda) = \begin{bmatrix} \cos \frac{\theta}{2} & -e^{i\lambda} \sin \frac{\theta}{2} \\ e^{i\phi} \sin \frac{\theta}{2} & e^{i(\phi+\lambda)} \cos \frac{\theta}{2} \end{bmatrix}.$$

## Widely used gates

There are several widely used quantum gates. Perhaps the most famous are the Pauli gates correspond to the Pauli matrices

$$I = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix},$$

$$X = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix},$$

$$Y = \begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix},$$

$$Z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}.$$

## Algebra basis

These gates form a basis for the algebra  $\mathfrak{su}(2)$ . Exponentiating them will thus give us a basis for  $SU(2)$ , the group within which all single-qubit gates live.

## Exponentiated Pauli gates

These exponentiated Pauli gates are called rotation gates  $R_\sigma(\theta)$  because they rotate the quantum state around the axis  $\sigma = X, Y, Z$  of the Bloch sphere by an angle  $\theta$ . They are defined as

$$R_X(\theta) = e^{-i\frac{\theta}{2}X} = \begin{bmatrix} \cos \frac{\theta}{2} & -i \sin \frac{\theta}{2} \\ -i \sin \frac{\theta}{2} & \cos \frac{\theta}{2} \end{bmatrix},$$

$$R_Y(\theta) = e^{-i\frac{\theta}{2}Y} = \begin{bmatrix} \cos \frac{\theta}{2} & -\sin \frac{\theta}{2} \\ \sin \frac{\theta}{2} & \cos \frac{\theta}{2} \end{bmatrix},$$

$$R_Z(\theta) = e^{-i\frac{\theta}{2}Z} = \begin{bmatrix} e^{-i\theta/2} & 0 \\ 0 & e^{i\theta/2} \end{bmatrix}.$$



## Basis for SU(2)

Because they form a basis for SU(2), any single-qubit gate can be decomposed into three rotation gates. Indeed

$$R_z(\phi)R_y(\theta)R_z(\lambda) = \begin{bmatrix} e^{-i\phi/2} & 0 \\ 0 & e^{i\phi/2} \end{bmatrix} \begin{bmatrix} \cos \frac{\theta}{2} & -\sin \frac{\theta}{2} \\ \sin \frac{\theta}{2} & \cos \frac{\theta}{2} \end{bmatrix} \begin{bmatrix} e^{-i\lambda/2} & 0 \\ 0 & e^{i\lambda/2} \end{bmatrix}$$

which we can rewrite as

$$e^{-i(\phi+\lambda)/2} \begin{bmatrix} \cos \frac{\theta}{2} & -e^{i\lambda} \sin \frac{\theta}{2} \\ e^{i\phi} \sin \frac{\theta}{2} & e^{i(\phi+\lambda)} \cos \frac{\theta}{2} \end{bmatrix},$$

which is, up to a global phase, equal to the expression for an arbitrary single-qubit gate.

## Two-Qubit Gates

A two-qubit gate is a physical action that is applied to two qubits. It can be represented by a matrix  $U$  from the group  $SU(4)$ . One important type of two-qubit gates are controlled gates, which work as follows: Suppose  $U$  is a single-qubit gate. A controlled- $U$  gate ( $CU$ ) acts on two qubits: a control qubit  $|x\rangle$  and a target qubit  $|y\rangle$ . The controlled- $U$  gate applies the identity  $I$  or the single-qubit gate  $U$  to the target qubit if the control gate is in the zero state  $|0\rangle$  or the one state  $|1\rangle$ , respectively.

## Control qubit

The control qubit is not acted upon. This can be represented as follows if

$$CU|xy\rangle = |xy\rangle \text{ if } |x\rangle = |0\rangle.$$

## In matrix form

It is easier to see in a matrix form. It can be written in matrix form by writing it as a superposition of the two possible cases, each written as a simple tensor product

$$CU = |0\rangle\langle 0| \otimes I + |1\rangle\langle 1| \otimes U = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & u_{00} & u_{01} \\ 0 & 0 & u_{10} & u_{11} \end{bmatrix}.$$

## CNOT gate

One of the most fundamental controlled gates is the CNOT gate. It is defined as the controlled- $X$  gate  $CX$ . It can be written in matrix form as follows:

$$\text{CNOT} = CX = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}.$$

## CX gate

It changes, when operating on a two-qubit state where the first qubit is the control qubit and the second qubit is the target qubit, the states (check this)

$$\text{CX}|00\rangle = |00\rangle,$$

$$\text{CX}|10\rangle = |11\rangle,$$

$$\text{CX}|01\rangle = |01\rangle,$$

$$\text{CX}|11\rangle = |10\rangle,$$

which you can easily see by simply multiplying the above matrix with any of the above states.

## Swap gate

A widely used two-qubit gate that goes beyond the simple controlled function is the SWAP gate. It swaps the states of the two qubits it acts upon

$$\text{SWAP}|xy\rangle = |yx\rangle.$$

and has the following matrix form

$$\text{SWAP} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

## Simple code

```
#!/usr/bin/env python
# coding: utf-8
import numpy as np
import qiskit as qk
from scipy.optimize import minimize

# # Initialize registers and circuit

n_qubits = 1 #Number of qubits
n_cbits = 1 #Number of classical bits (the number of qubits you want t
qreg = qk.QuantumRegister(n_qubits) #Create a quantum register
creg = qk.ClassicalRegister(n_cbits) #Create a classical register
circuit = qk.QuantumCircuit(qreg,creg) #Create your quantum circuit

circuit.draw() #Draw circuit. It is empty
```

Thereafter we perform operations on qubit

```
circuit.x(qreg[0]) #Applies a Pauli X gate to the first qubit in the q
circuit.draw()
```

and select a qubit to measure and encode the results to a classical bit

```
#Measure the first qubit in the quantum register
#and encode the results to the first qubit in the classical register
```



## Exercises on Bell states

```
import qiskit as qk
from qiskit import QuantumRegister, ClassicalRegister, QuantumCircuit
from qiskit.visualization import plot_histogram
from qiskit_aer import AerSimulator
import matplotlib.pyplot as plt
```

### Setting up a circuit

```
qreg_q = QuantumRegister(2) # 2 qubits
creg_c = ClassicalRegister(2) # 2 classical bits

qc = QuantumCircuit(qreg_q, creg_c) # alternatively, circuit = QuantumCircuit(qreg_q, creg_c)

qc.h(qreg_q[0]) # apply the hadamard gate to the first qubit (to the rest of the qubits)

# apply the CNOT gate with the first qubit being the control qubit and the second qubit being the target qubit
qc.cx(qreg_q[0], qreg_q[1])

# measure the qubits, specify which qubit you want to measure
qc.measure(qreg_q[0], creg_c[0])
qc.measure(qreg_q[1], creg_c[1])
```

Next we will use an ideal(noiseless) simulator (AerSimulator) to run our circuit. We will use the qasm\_simulator backend. This simulator returns a result object which contains the counts, or

# Testing the Bell states

```
from qiskit import QuantumCircuit, transpile, assemble, IBMQ
from qiskit.visualization import circuit_drawer
import qiskit_aer
import matplotlib.pyplot as plt

# Create a quantum circuit with two qubits
bell_circuit = QuantumCircuit(2, 2)

# Apply Hadamard gate to the first qubit
bell_circuit.h(0)

# Apply a CNOT gate with the first qubit as the control and the second
bell_circuit.cx(0, 1)

# Add measurements to the circuit
bell_circuit.measure([0, 1], [0, 1])

# Visualize the circuit
print("Quantum Circuit:")
print(bell_circuit)
```

# Running the calculations

```
# Number of shots
num_shots = 10000

# Simulate the circuit using the Aer simulator
simulator = qiskit_aer.Aer.get_backend('qasm_simulator')

# Transpile the circuit for the simulator
transpiled_circuit = transpile(bell_circuit, simulator)

# Execute the transpiled circuit on the simulator with the specified n
result = simulator.run(transpiled_circuit, shots=num_shots).result()

# Get measurement counts
counts = result.get_counts(bell_circuit)

# Get and print the measurement results
counts = result.get_counts(bell_circuit)
print("\nMeasurement Results:")
print(counts)
```

Then plotting the histogram over counts

```
# Plot the histogram using Matplotlib  
plt.bar(counts.keys(), counts.values())  
plt.title('Bell State Measurement Results')  
plt.ylabel('Counts')  
plt.show()
```

# Getting serious, Quantum Simulator with Hardware noise model

```
from qiskit import QuantumCircuit, transpile, assemble, Aer, IBMQ
from qiskit.visualization import plot_histogram
from qiskit.providers.aer.noise import NoiseModel

# you will not be able to run unless you provide your token here
api_token = 'your_api_token'
# Load your IBM Quantum account
IBMQ.save_account(api_token, overwrite=True) # This stores your credentials
IBMQ.load_account()
# Get the least busy backend
provider = IBMQ.get_provider(hub='ibm-q', group='open', project='main')
#backend = provider.get_backend('your_preferred_backend')
backend = provider.get_backend('ibm_osaka')

# Get the noise model for the selected backend
noise_model = NoiseModel.from_backend(backend)

# Create a quantum circuit with two qubits
bell_circuit = QuantumCircuit(2, 2)

# Apply Hadamard gate to the first qubit
bell_circuit.h(0)

# Apply a CNOT gate with the first qubit as the control and the second
bell_circuit.cx(0, 1)
```

# Then run and visualize it

```
# Number of shots
```

```
num_shots = 10000
```

```
# Simulate the circuit using the Aer simulator with the noise model
```

```
simulator = Aer.get_backend('qasm_simulator')
```

```
noisy_transpiled_circuit = transpile(bell_circuit, simulator, optimization_level=3)
```

```
noisy_circuit = assemble(noisy_transpiled_circuit, shots=num_shots, noise_model=noise_model)
```

```
# Execute the noisy circuit on the simulator
```

```
result = simulator.run(noisy_circuit).result()
```

```
# Get and print the measurement results
```

```
counts = result.get_counts(bell_circuit)
```

```
print("\nMeasurement Results:")
```

```
print(counts)
```

```
# Plot the histogram using Matplotlib
```

```
plot_histogram(counts, title='Bell State Measurement Results')
```

## And without the noise model

```
# Provide your IBM Quantum Experience API token here
api_token = 'your_api_token'
# Load your IBM Quantum account
IBMQ.save_account(api_token, overwrite=True) # This stores your credentials
IBMQ.load_account()

# Get the least busy backend
#provider = IBMQ.get_provider(hub='your_hub', group='your_group', project='your_project')
provider = IBMQ.get_provider(hub='ibm-q', group='open', project='main')
#backend = provider.get_backend('your-preferred-backend')
backend = provider.get_backend('ibm_osaka')

# Create a quantum circuit with two qubits
bell_circuit = QuantumCircuit(2, 2)

# Apply Hadamard gate to the first qubit
bell_circuit.h(0)

# Apply a CNOT gate with the first qubit as the control and the second qubit as the target
bell_circuit.cx(0, 1)

# Add measurements to the circuit
bell_circuit.measure([0, 1], [0, 1])

# Transpile the circuit for the chosen backend
transpiled_circuit = transpile(bell_circuit, backend)

# Execute the transpiled circuit on the IBM Quantum computer
```

## Hamiltonians, one qubit system

We start with a simple  $2 \times 2$  Hamiltonian matrix expressed in terms of Pauli  $X$  and  $Z$  matrices, as discussed in the project text as well. We define a symmetric matrix  $H \in \mathbb{R}^{2 \times 2}$

$$H = \begin{bmatrix} H_{11} & H_{12} \\ H_{21} & H_{22} \end{bmatrix},$$



## Rewriting the Hamiltonian

We let  $H = H_0 + H_I$ , where

$$H_0 = \begin{bmatrix} E_1 & 0 \\ 0 & E_2 \end{bmatrix},$$

is a diagonal matrix. Similarly,

$$H_I = \begin{bmatrix} V_{11} & V_{12} \\ V_{21} & V_{22} \end{bmatrix},$$

where  $V_{ij}$  represent various interaction matrix elements. We can view  $H_0$  as the non-interacting solution

$$H_0|0\rangle = E_1|0\rangle, \tag{2}$$

and

$$H_0|1\rangle = E_2|1\rangle, \tag{3}$$

where we have defined the orthogonal computational one-qubit basis states  $|0\rangle$  and  $|1\rangle$ .

## Using Pauli matrices

We rewrite  $H$  (and  $H_0$  and  $H_I$ ) via Pauli matrices

$$H_0 = \mathcal{E}I + \Omega\sigma_z, \quad \mathcal{E} = \frac{E_1 + E_2}{2}, \quad \Omega = \frac{E_1 - E_2}{2},$$

and

$$H_I = cI + \omega_z\sigma_z + \omega_x\sigma_x,$$

with  $c = (V_{11} + V_{22})/2$ ,  $\omega_z = (V_{11} - V_{22})/2$  and  $\omega_x = V_{12} = V_{21}$ .  
We let our Hamiltonian depend linearly on a strength parameter  $\lambda$

$$H = H_0 + \lambda H_I,$$

with  $\lambda \in [0, 1]$ , where the limits  $\lambda = 0$  and  $\lambda = 1$  represent the non-interacting (or unperturbed) and fully interacting system, respectively. The model is an eigenvalue problem with only two available states.

## Selecting parameters

Here we set the parameters  $E_1 = 0$ ,  $E_2 = 4$ ,  $V_{11} = -V_{22} = 3$  and  $V_{12} = V_{21} = 0.2$ .

The non-interacting solutions represent our computational basis. Pertinent to our choice of parameters, is that at  $\lambda \geq 2/3$ , the lowest eigenstate is dominated by  $|1\rangle$  while the upper is  $|0\rangle$ . At  $\lambda = 1$  the  $|0\rangle$  mixing of the lowest eigenvalue is 1% while for  $\lambda \leq 2/3$  we have a  $|0\rangle$  component of more than 90%. The character of the eigenvectors has therefore been interchanged when passing  $z = 2/3$ . The value of the parameter  $V_{12}$  represents the strength of the coupling between the two states.

## Setting up the matrix

```
from matplotlib import pyplot as plt
import numpy as np
dim = 2
Hamiltonian = np.zeros((dim,dim))
e0 = 0.0
e1 = 4.0
Xnondiag = 0.20
Xdiag = 3.0
Eigenvalue = np.zeros(dim)
# setting up the Hamiltonian
Hamiltonian[0,0] = Xdiag+e0
Hamiltonian[0,1] = Xnondiag
Hamiltonian[1,0] = Hamiltonian[0,1]
Hamiltonian[1,1] = e1-Xdiag
# diagonalize and obtain eigenvalues, not necessarily sorted
EigValues, EigVectors = np.linalg.eig(Hamiltonian)
permute = EigValues.argsort()
EigValues = EigValues[permute]
# print only the lowest eigenvalue
print(EigValues[0])
```

Now rewrite it in terms of the identity matrix and the Pauli matrix X and Z

```
# Now rewrite it in terms of the identity matrix and the Pauli matrix
X = np.array([[0,1],[1,0]])
Y = np.array([[0,-1j],[1j,0]])
Z = np.array([[1,0],[0,-1]])
```

## Where are we going?

For a one-qubit system we can reach every point on the Bloch sphere (as discussed earlier) with a rotation about the  $x$ -axis and the  $y$ -axis.

We can express this mathematically through the following operations (see whiteboard for the drawing), giving us a new state  $|\psi\rangle$

$$|\psi\rangle = R_y(\phi)R_x(\theta)|0\rangle.$$

## Possible ansatzes

We can produce multiple ansatzes for the new state in terms of the angles  $\theta$  and  $\phi$ . With these ansatzes we can in turn calculate the expectation value of the above Hamiltonian, now rewritten in terms of various Pauli matrices (and thereby gates), that is compute

$$\langle\psi|(c + \mathcal{E})I + (\Omega + \omega_z)\sigma_z + \omega_x\sigma_x|\psi\rangle.$$

We can now set up a series of ansatzes for  $|\psi\rangle$  as function of the angles  $\theta$  and  $\phi$  and find thereafter the variational minimum using for example a gradient descent method.

## More on rotation operators

To do so, we need to remind ourselves about the mathematical expressions for the rotational matrices/operators.

$$R_x(\theta) = \cos \frac{\theta}{2} \mathbf{I} - i \sin \frac{\theta}{2} \boldsymbol{\sigma}_x,$$

and

$$R_y(\phi) = \cos \frac{\phi}{2} \mathbf{I} - i \sin \frac{\phi}{2} \boldsymbol{\sigma}_y.$$

## Code example

```
# define the rotation matrices
# Define angles theta and phi
theta = 0.5*np.pi; phi = 0.2*np.pi
Rx = np.cos(theta*0.5)*I-1j*np.sin(theta*0.5)*X
Ry = np.cos(phi*0.5)*I-1j*np.sin(phi*0.5)*Y
#define basis states
basis0 = np.array([1,0])
basis1 = np.array([0,1])

NewBasis = Ry @ Rx @ basis0
print(NewBasis)
# Compute the expectation value
#Note hermitian conjugation
Energy = NewBasis.conj().T @ Hamiltonian @ NewBasis
print(Energy)
```

Not an impressive results. We set up now a loop over many angles  $\theta$  and  $\phi$  and compute the energies

```
# define a number of angles
n = 20
angle = np.arange(0,180,10)
n = np.size(angle)
ExpectationValues = np.zeros((n,n))
for i in range (n):
    theta = np.pi*angle[i]/180.0
    Rx = np.cos(theta*0.5)*I-1j*np.sin(theta*0.5)*X
    for j in range (n):
```



## Topics next week

1. We will extend the above one-qubit Hamiltonian to a two-qubit problem and analyze how we can set up its simulation
2. Before we introduce the Variational Quantum Eigensolver (VQE), we need to discuss entropy as a measure of entanglement
3. If we get time, we start our discussion of the VQE algorithm

## Exercises this week: Hamiltonians and project 1

As an initial test, we consider a simple  $2 \times 2$  real Hamiltonian consisting of a diagonal part  $H_0$  and off-diagonal part  $H_I$ , playing the roles of a non-interactive one-body and interactive two-body part respectively. Defined through their matrix elements, we express them in the Pauli basis  $|0\rangle$  and  $|1\rangle$

$$H = H_0 + H_I$$
$$H_0 = \begin{bmatrix} E_1 & 0 \\ 0 & E_2 \end{bmatrix}, \quad H_I = \lambda \begin{bmatrix} V_{11} & V_{12} \\ V_{21} & V_{22} \end{bmatrix}$$

Where  $\lambda \in [0, 1]$  is a coupling constant parameterizing the strength of the interaction.

## Rewriting in terms of Pauli matrices

Define

$$E_+ = \frac{E_1 + E_2}{2}, \quad E_- = \frac{E_1 - E_2}{2}$$

show that by combining the identity and  $Z$  Pauli matrix, this can be expressed as

$$H_0 = E_+ I + E_- Z$$

## The interaction part

For  $H_I$  we use the same trick to fill the diagonal, defining  $V_+ = (V_{11} + V_{22})/2$ ,  $V_- = (V_{11} - V_{22})/2$ . From the hermiticity requirements of  $H$ , we note that  $V_{12} = V_{21} \equiv V_o$ . Use this to simplify the problem to a simple  $X$  term.

$$H_I = V_+ I + V_- Z + V_o X$$

## Measurement basis

For the above system show that the Pauli  $X$  matrix can be rewritten in terms of the Hadamard matrices and the Pauli  $Z$  matrix, that is

$$X = HZH.$$